

Phase 5: Project Development

This phase documents how LegalEase was built, step by step, following Milestones 1 to 5 of the project workflow.

5.1 Environment Setup (Milestone 1)

```
# 1. Verify Python 3.10+
python --version

# 2. Create and activate a virtual environment
python -m venv venv
venv\Scripts\activate           # Windows
# source venv/bin/activate      # Linux / macOS

# 3. Install dependencies
pip install fastapi uvicorn
pip install streamlit
pip install google-generativeai
pip install python-docx fpdf Pillow requests python-dotenv

# 4. Freeze dependencies
pip freeze > requirements.txt
```

.env (never commit this file):

```
GEMINI_API_KEY=your_gemini_api_key_here
```

.gitignore:

```
.env
```

```
venv/  
__pycache__/  
*.pyc
```

Model selection: gemini-1.5-pro was chosen for its long context window (up to 1M tokens), multi-part structured prompting and strong formatting/reasoning for formal documents. The model name is kept in config.py so it can be updated if Google retires or renames models.

5.2 Configuration: config.py

```
import os  
from dotenv import load_dotenv  
  
load_dotenv()  
  
BASE_DIR = os.path.dirname(os.path.abspath(__file__))  
  
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")  
MODEL_NAME = os.getenv("GEMINI_MODEL", "gemini-1.5-pro")  
BACKEND_URL = os.getenv("BACKEND_URL", "http://localhost:  
  
WEB_LOGO_PATH = os.path.join(BASE_DIR, "Image", "inverseL  
DOC_LOGO_PATH = os.path.join(BASE_DIR, "Image", "Logo.png")  
FOOTER_TEXT = "LegalEase Inc. | contact@legalease.com | A
```

5.3 AI Core: ai_core/ gemini_generator.py (Milestone 2)

```
import google.generativeai as genai  
from config import GEMINI_API_KEY, MODEL_NAME
```

```

class GeminiDocumentGenerator:
    def __init__(self, model_name: str = MODEL_NAME):
        genai.configure(api_key=GEMINI_API_KEY)
        self.model = genai.GenerativeModel(model_name)

    def generate_document(self, document_type, parties, titles, dates, terms):
        prompt = (
            f"Generate a comprehensive legal document titled '{document_type}'.\n"
            f"Involved parties: {parties}\n"
            f"Effective Date: {dates}\n"
            f"Terms and conditions: {terms}\n"
            "Ensure formal legal structure with multiple sections.\n"
            "Use numbered section headings in plain text format."
        )
        response = self.model.generate_content(prompt)
        return response.text

```

5.4 Backend (Milestone 3)

legalEaseAPI/main.py

```

from fastapi import FastAPI
from legalEaseAPI.routes import router

app = FastAPI(title="LegalEase - AI Legal Document Generator")
app.include_router(router)

@app.get("/")
def home():
    return {"message": "Welcome to LegalEase AI Legal Document Generator"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

legalEaseAPI/routes.py

```
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from ai_core.gemini_generator import GeminiDocumentGenerator

router = APIRouter()
gemini_generator = GeminiDocumentGenerator()

class DocumentRequest(BaseModel):
    document_type: str
    parties: str
    terms: str
    dates: str

@router.post("/generate")
def generate_legal_document(request: DocumentRequest):
    try:
        response = gemini_generator.generate_document(
            request.document_type,
            request.parties,
            request.terms,
            request.dates,
        )
        return {"document": response}
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Doc
```

Run and verify:

```
uvicorn legalEaseAPI.main:app --reload
# open http://127.0.0.1:8000/docs and test POST /generate
```

5.5 Formatting Utilities: ai_core/

generator.py (Milestone 2 and 4)

```
import html
import os
from io import BytesIO

from docx import Document
from docx.enum.text import WD_ALIGN_PARAGRAPH
from docx.shared import Inches, Pt
from fpdf import FPDF

from config import DOC_LOGO_PATH, FOOTER_TEXT

_REPLACEMENTS = {
    "\u2018": "'", "\u2019": "'", "\u201c": '"', "\u201d": '"',
    "\u2013": "-", "\u2014": "-", "\u2022": "-", "\u2026": "..."
}

def sanitize_text(text: str) -> str:
    """Replace typographic characters and strip markdown
    for old, new in _REPLACEMENTS.items():
        text = text.replace(old, new)
    return text.replace("**", "")

def _clean_line(line: str) -> str:
    s = line.strip()
    if s.startswith("#"):
        s = s.lstrip("#").strip()
    return s

def _is_heading(s: str) -> bool:
    return s.endswith(":") and len(s) < 80

def format_html_preview(text: str) -> str:
```

```

blocks = []
for line in text.split("\n"):
    s = _clean_line(line)
    if not s:
        continue
    safe = html.escape(s)
    if _is_heading(s):
        blocks.append(f"<h4 style='margin:14px 0 4px'"
    else:
        blocks.append(f"<p style='margin:4px 0'>{safe}
return "".join(blocks)

```

```

def format_docx(text: str, doc_type: str, terms: str | No
doc = Document()
normal = doc.styles["Normal"]
normal.font.name = "Times New Roman"
normal.font.size = Pt(12)

if os.path.exists(DOC_LOGO_PATH):
    p = doc.add_paragraph()
    p.alignment = WD_ALIGN_PARAGRAPH.CENTER
    p.add_run().add_picture(DOC_LOGO_PATH, width=Inch

title = doc.add_heading(doc_type.title(), level=1)
title.alignment = WD_ALIGN_PARAGRAPH.CENTER

for line in text.split("\n"):
    s = _clean_line(line)
    if not s:
        continue
    para = doc.add_paragraph()
    run = para.add_run(s)
    run.bold = _is_heading(s)

if terms: # auto-generated terms table from semicol
    items = [t.strip() for t in terms.split(";") if t
    if items:
        doc.add_heading("Summary of Key Terms", level
        table = doc.add_table(rows=1, cols=2)

```

```

        table.style = "Table Grid"
        table.rows[0].cells[0].text = "No."
        table.rows[0].cells[1].text = "Term"
        for i, item in enumerate(items, 1):
            cells = table.add_row().cells
            cells[0].text = str(i)
            cells[1].text = item

    footer = doc.sections[0].footer.paragraphs[0]
    footer.text = FOOTER_TEXT
    footer.alignment = WD_ALIGN_PARAGRAPH.CENTER

    buffer = BytesIO()
    doc.save(buffer)
    return buffer.getvalue()

class _LegalPDF(FPDF):
    def __init__(self, title: str):
        super().__init__()
        self.doc_title = title

    def header(self):
        if os.path.exists(DOC_LOGO_PATH):
            self.image(DOC_LOGO_PATH, x=85, y=8, w=40)
        self.set_y(30)
        self.set_font("Arial", "B", 12)
        self.cell(0, 8, self.doc_title, 0, 1, "C")
        self.ln(4)

    def footer(self):
        self.set_y(-15)
        self.set_font("Arial", "I", 8)
        self.cell(0, 10, FOOTER_TEXT, 0, 0, "C")

def _latin1(s: str) -> str:
    return s.encode("latin-1", "replace").decode("latin-1")

```

```
def format_pdf(text: str, doc_type: str) -> bytes:
    pdf = _LegalPDF(_latin1(doc_type))
    pdf.set_auto_page_break(auto=True, margin=20)
    pdf.add_page()
    for line in text.split("\n"):
        s = _clean_line(line)
        if not s:
            continue
        pdf.set_font("Arial", "B" if _is_heading(s) else
        pdf.multi_cell(0, 6, _latin1(s))
        pdf.ln(1)
    return pdf.output(dest="S").encode("latin-1")    # fpdf
```

Why `sanitize_text` and `_latin1`? Classic FPDF core fonts only support Latin-1. Typographic quotes and dashes returned by the AI would otherwise crash or corrupt PDF generation.

5.6 Frontend: `frontend/app.py` (Milestone 4)

```
import os
import sys

import requests
import streamlit as st

sys.path.append(os.path.dirname(os.path.dirname(os.path.a

from ai_core.generator import format_docx, format_html_pr
from config import BACKEND_URL, WEB_LOGO_PATH

st.set_page_config(page_title="LegalEase", layout="center

# Logo and title
col1, col2, col3 = st.columns([1, 2, 1])
with col2:
```

```

        st.image(WEB_LOGO_PATH, use_container_width=True)
st.markdown("<h2 style='text-align: center;'>AI Legal Doc

# Inputs
document_type = st.text_input("Document Type (Ex: Agreeeme
parties = st.text_area("Parties Involved")
terms = st.text_area("Terms & Conditions (Use semicolons
dates = st.text_input("Effective Date")

# Generate
if st.button("Generate Document"):
    if not all(v.strip() for v in (document_type, parties
        st.warning("Please fill in all fields.")
    else:
        with st.spinner("Generating your document..."):
            try:
                response = requests.post(
                    f"{BACKEND_URL}/generate",
                    json={"document_type": document_type,
                        "terms": terms, "dates": dates}
                    timeout=120,
                )
                response.raise_for_status()
                st.session_state.generated_text = sanitiz
                st.session_state.doc_type = document_type
                st.session_state.terms = terms
                st.session_state.show_edit = False
                st.success("Document Generated Successful
            except requests.RequestException as e:
                st.error(f"Could not generate the documen
else:
    if "generated_text" not in st.session_state:
        st.info("Click 'Generate Document' to start")

# Preview, edit, download
if "generated_text" in st.session_state:
    styled_html = format_html_preview(st.session_state.ge
    st.markdown(
        "<div style='background:#111827;color:#e5e7eb;pad
        f"max-height:300px;overflow-y:auto'>{styled_html}

```

```

        unsafe_allow_html=True,
    )

    if st.button("📄 Click to Edit Document"):
        st.session_state.show_edit = True

    if st.session_state.get("show_edit"):
        st.session_state.generated_text = st.text_area(
            "Edit Document Below:", st.session_state.generated_text
        )

    text = st.session_state.generated_text
    doc_type = st.session_state.doc_type
    file_stem = doc_type.replace(" ", "_").lower()

    st.download_button("📄 Download as .TXT", data=text,
        st.download_button(
            "📄 Download as .DOCX",
            data=format_docx(text, doc_type, st.session_state.generated_text),
            file_name=f"{file_stem}.docx",
            mime="application/vnd.openxmlformats-officedocument
        )
    st.download_button(
        "📄 Download as .PDF",
        data=format_pdf(text, doc_type),
        file_name=f"{file_stem}.pdf",
        mime="application/pdf",
    )

```

5.7 Running the Application (Milestone 5)

run.sh

```

#!/bin/bash
uvicorn legalEaseAPI.main:app --reload &
streamlit run frontend/app.py

```

Manual run (two terminals, venv activated in both):

```
uvicorn legalEaseAPI.main:app --reload          # http://127.0.0.1:8000/
streamlit run frontend/app.py                  # http://localhost:8501/
```

5.8 Development Notes and Decisions

Decision	Reason
Separate frontend and backend	Clean separation, API can be reused by other clients
Formatting done in <code>ai_core/generator.py</code>	Keeps AI logic and file generation independent
<code>st.session_state</code> for generated text	Streamlit reruns the script on every interaction; state preserves the document during edit/download
Prompt asks for plain-text headings	Reduces stray markdown (##) in DOCX/PDF outputs
Model name and URLs in <code>config.py</code>	Easy to change without touching logic

5.9 Development Progress Summary

Milestone	Status
M1 Model selection, architecture, environment	☒ Done
M2 Core functionalities (generation, formatting)	☒ Done
M3 API logic (<code>main.py</code> , <code>routes.py</code>)	☒ Done

M4 Frontend (Streamlit UI, preview, edit, downloads)	☒ Done
M5 Local deployment and verification	☒ Done